

Documentacion Tecnica

Sistema Experto de Cotizacion - Maderas Gerardo

Munoz y Girata - Inteligencia Artificial II - Los Libertadores, 2026

1. Introduccion

1.1 Descripcion del dominio

Maderas Gerardo es un taller de carpinteria artesanal ubicado en Bogota, Colombia, en operacion desde 1998. Fabrica muebles a medida: puertas, closets, cocinas integrales, camas, mesas y estantes, trabajando con MDF, triplex, madera solida y pino. El proceso de cotizacion, que dependia de la experiencia del carpintero, se formalizo en un sistema experto con 40 reglas de produccion.

1.2 Justificacion

La cotizacion manual es inconsistente: el mismo mueble puede recibir precios diferentes segun quien cotiche. Un sistema experto elimina esa variabilidad aplicando reglas formales y reproducibles. Permite que cualquier persona obtenga cotizaciones precisas con explicacion del razonamiento.

1.3 Alcance

Cubre 6 tipos de muebles, 4 materiales, 4 acabados, 8 extras opcionales, recargos logísticos y advertencias de compatibilidad. No cubre fabricacion ni seguimiento post-venta.

2. Diseno del sistema

2.1 Arquitectura

Arquitectura cliente-servidor: backend FastAPI (Python) con motor de inferencia + frontend React (JavaScript). Comunicacion via API REST con JSON. El backend se organiza en capas: main.py (entrada), config.py (constantes), models/ (esquemas), routers/ (HTTP), services/ (logica), knowledge/ (motor + reglas).

2.2 Flujo de datos

1) Usuario selecciona tipo, material, color y extras. 2) Frontend envia POST a /api/cotizar. 3) Servicio transforma datos y los pasa al motor. 4) Motor aplica encadenamiento adelante hasta punto fijo. 5) Resultado (precio, tiempo, reglas, advertencias) vuelve como JSON. 6) Frontend muestra resultado con desglose y explicacion.

3. Base de conocimiento

3.1 Predicados

`es_puerta(x)`, `es_closet(x)`, `es_cocina(x)`, `es_cama(x)`, `es_mesa(x)`, `es_estante(x)`
`material_mdf(x)`, `material_triplex(x)`, `material_solida(x)`, `material_pino(x)`
`tiene_cajones(x)`, `tiene_vidrio(x)`, `tiene_espejo(x)`, `puertas_corredizas(x)`
`es_urgente(x)`, `requiere_instalacion(x)`, `fuera_de_ciudad(x)`, `es_empotrado(x)`
`precio_muy_economico(x)`, `precio_economico(x)`, `precio_medio(x)`, `precio_alto(x)`

3.2 Reglas (40 total, 4 categorias)

Categoria A (24 reglas): Clasificacion base por tipo x material. Ejemplo: Si `es_puerta(x)` AND `material_mdf(x)` ENTONCES `precio_muy_economico(x)` [CF: 0.95]

Categoria B (8 reglas): Modificadores por acabado sobre precio base.

Categoria C (5 reglas): Recargos por complejidad y extras.

Categoria D (3 reglas): Logistica (urgencia +20%, instalacion, fuera de ciudad).

4. Motor de inferencia

4.1 Encadenamiento hacia adelante

Forward chaining con detección de punto fijo. Cada iteración recorre las 40 reglas, evalúa condiciones contra hechos actuales, y agrega conclusiones como nuevos hechos. Se repite hasta que no se activen reglas nuevas.

4.2 Funciones de explicación

`explicar_razonamiento()`: Genera texto con la cadena completa de reglas aplicadas.

`porque(conclusion)`: Traza desde una conclusión hasta los hechos iniciales que la provocaron.

4.3 Casos especiales

Sin conclusión: devuelve resultado vacío con mensaje. Ciclos: detección de punto fijo los evita. Logging: cada regla activada se registra con sus condiciones y conclusión.

5. Implementacion tecnica

Backend: Python 3.11, FastAPI, Uvicorn, Pydantic. Frontend: React, Vite, JavaScript, Tailwind CSS, lucide-react, jsPDF. Se eligio FastAPI + React sobre Streamlit por arquitectura mas profesional con separacion backend/frontend. Motor en Python puro sin dependencias externas.

Desafios

EmailJS no permite destinatarios dinamicos en plan gratuito; se migro a WhatsApp. Estado del sidebar afectaba cotizador; se aislo con estado local. Constantes centralizadas en config con variables de entorno.

6. Validacion

5 casos de prueba documentados en tests/casos_prueba.md. Cubren: caso basico (1 regla), caso complejo (múltiples reglas), caso sin conclusion, caso con advertencia, y caso de borde (todos los extras). Todos pasaron satisfactoriamente.

7. Conclusiones

Se construyo un sistema experto funcional con 40 reglas, motor de inferencia, interfaz web profesional, PDF y WhatsApp. Basado en un dominio real. La formalizacion del conocimiento requiere iteracion con el experto. Mejoras futuras: configurador 3D, aprendizaje automatico para ajustar certezas, integracion con inventario.